

FORGE — FDE ACADEMY · MULTI-AGENT ORCHESTRATION · ENTERPRISE AI

Multi-Agent Orchestration

Framework Choice

Chaos Drill

Orchestrator / sub-agent · parallel agents · handoffs · CrewAI vs AutoGen vs Swarm · Azure Service Bus eventing · resilience under failure

orchestrator / sub-agent

handoffs

Azure Service Bus

verifier agent

chaos drill

SESSION MAP

What we cover today



Multi-agent patterns on Azure

Orchestrator / sub-agent · parallel agents · handoffs · state-sharing

1



Framework choice

CrewAI · AutoGen → Agent Framework · OpenAI Swarm → Agents SDK

2



Azure eventing & scale-out

Service Bus queues, topics, sessions · competing consumers · dead-letter

3



Failure scenarios

Cascades · drift · duplicate work · poison messages · runaway cost

4



Chaos Drill

Inject agent failures on purpose · verify graceful degradation

5



Two industry case studies

CPG demand planning (+ verifier) · KPMG multi-agent audit

6

OUTCOMES

By the end, you will be able to...



Pick a topology

Choose orchestrator/sub-agent, parallel, or handoff for a given task — and know why.



Choose a framework

Justify CrewAI vs Microsoft Agent Framework vs OpenAI Agents SDK for the job.



Wire agents on Azure

Use Service Bus queues, topics and sessions to decouple and scale agents out.



Design safe handoffs

Share state cleanly and add a verifier agent to catch bad output before it acts.



Anticipate failure

Name the ways multi-agent systems break and the mitigation for each.



Run a chaos drill

Inject agent failures deliberately and prove the system degrades gracefully.

01

MULTI-AGENT PATTERNS ON AZURE

Orchestrator, parallel & handoffs



FOUNDATIONS

Why more than one agent?



A single agent

- One prompt tries to do everything — plan, fetch, model, check.
- Context bloats; the model loses focus on long tasks.
- No second opinion — one hallucination becomes the answer.



A team of agents

- Each agent owns one bounded, testable job.
- Specialists run in parallel; an orchestrator reconciles.
- A verifier can challenge results before anything acts.

Why the industry is moving here



A Cornell study found multi-agent teams solved complex planning tasks far more often than a lone agent ($\approx 43\%$ vs $\approx 3\%$).



Gartner logged a $\sim 1,445\%$ jump in multi-agent enquiries in a year, and expects $\sim 40\%$ of enterprise apps to use task-specific agents by end-2026.

THE MAP

Three ways agents work together



Orchestrator / sub-agent

A supervisor plans, delegates to specialist workers, and synthesises their results.

Central control · auditable



Parallel agents

Independent agents run at once on the same input; a consensus/aggregator merges them.

Speed · diverse opinions



Handoffs

An agent transfers the whole conversation to a peer better suited to the next step.

Fluid · specialist routing

Most real systems combine all three: an orchestrator that fans out to parallel workers, with handoffs to specialists at the edges.

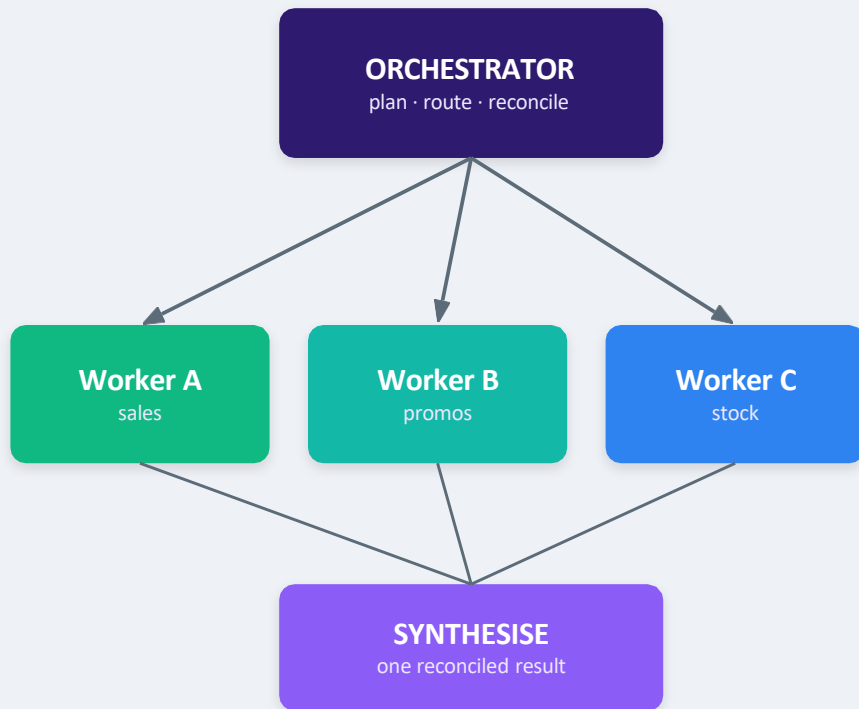
PATTERN 1

Orchestrator / sub-agent (supervisor–worker)

How it works

- The orchestrator interprets the goal and decomposes it (a work-breakdown).
- It routes each sub-task to a specialist worker and monitors progress.
- Workers return structured output; the orchestrator reconciles and synthesises.
- Keep the orchestrator thin — route & synthesise, not heavy reasoning.

Think: project manager + WBS + a team of specialists.



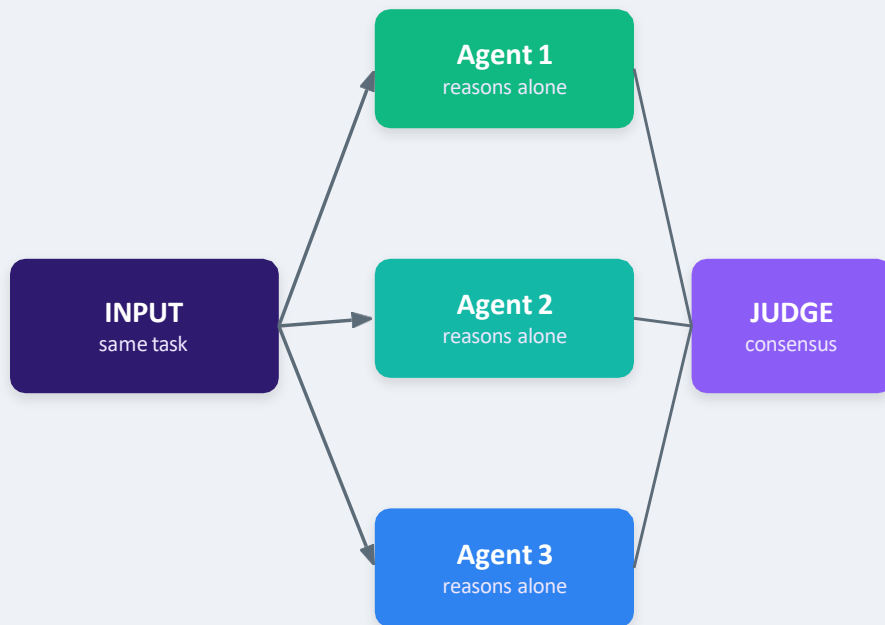
PATTERN 2

Parallel agents (fan-out / fan-in & debate)

How it works

- Independent agents run at the same time on the same input (fan-out).
- Results merge at an aggregator / consensus node (fan-in).
- Debate variant: give each agent a distinct role — optimist, critic, risk — so blind spots surface.
- A verifier or judge picks the final answer; disagreement is a feature, not a bug.

Consensus: majority vote · confidence-weighted · judge agent.



PATTERN 3

Handoffs — transfer control to a specialist



How it works

- An agent decides it isn't the right one, and hands the whole conversation to a peer.
- Popularised by OpenAI Swarm: a handoff is just a tool that returns another agent.
- The new agent inherits the history but swaps in its own instructions and tools.
- Great for triage → specialist routing; watch drift after ~8–10 chained handoffs.

Swarm is deprecated → the OpenAI Agents SDK carries the handoff model.

handoff.py

```
# OpenAI Agents SDK — handoff = agent-as-tool
from agents import Agent, Runner

refunds = Agent(name='Refunds',
                 instructions='Process refunds only.')

triage = Agent(
    name='Triage',
    instructions='Route the customer.',
    handoffs=[refunds], # transfer target
)

# triage may hand the whole convo to refunds
Runner.run_sync(triage, 'I want my money back')
```

CONTRACTS

Handoff & state-sharing patterns



Direct shared state

Workers read inputs from and write outputs to agreed fields on one shared state object.



Typed message channels

Append to a typed list with a reducer (e.g. `add_messages`) so nothing is overwritten.



Structured output

Each worker returns a typed dict that maps to specific state fields — parseable, validated.



Handoff messages

The supervisor writes a task spec to state; the worker reads it and executes against it.



Anti-patterns

Overloading the supervisor with reasoning · workers with overlapping responsibilities · workers steering control flow directly · ambiguous, untyped messages between agents.

02

FRAMEWORK CHOICE

CrewAI · AutoGen · OpenAI Swarm



THE FIELD

Three families — and where each is headed



CrewAI

Role-based crews

Agents as a team of roles with tasks; sequential or hierarchical process. Fast to stand up, business-friendly.

Active · standalone



AutoGen

→ Microsoft Agent Framework

Pioneered group-chat multi-agent. AutoGen + Semantic Kernel have converged into Agent Framework (GA 2026).

AutoGen: maintenance mode



OpenAI Swarm

→ OpenAI Agents SDK

Introduced the lean handoff model. Swarm is deprecated; the Agents SDK is the production successor.

Swarm: deprecated

LangGraph sits alongside these as the graph-based option when you need maximum control — it's the reference in the comparison ahead.

CREWAI

Think in roles, not nodes



Agent

role · goal · backstory · tools



Task

a description + expected output



Crew

agents + tasks; sequential or hierarchical



Flow

orchestration: state, events, branching

Where it shines

- Task maps naturally to a team of specialists (researcher → writer → editor).
- Business users can configure agents/tasks in YAML — little code.
- Hub-and-spoke: agents route through the orchestrator, not peer-to-peer — auditable.
- LLM-agnostic: a different model per agent (cost vs quality).
- Per-agent memory (short-term buffer + long-term vector recall); async tasks.
- Standalone, no LangChain dependency; reported ~5.76x faster than LangGraph in a vendor benchmark.

AUTOGEN → MICROSOFT AGENT FRAMEWORK

The Microsoft path

What happened

- AutoGen (from MS Research) pioneered group-chat, debate and reflection between agents.
- In Oct 2025 AutoGen + Semantic Kernel converged into Microsoft Agent Framework; v1.0 shipped in 2026.
- AutoGen is now maintenance-only — new work targets Agent Framework.
- Agent Framework keeps AutoGen's simple agent/group-chat model and adds SK's enterprise features.

[Native path to Azure AI Foundry Agent Service.](#)

What Agent Framework gives you

- One SDK across Python and .NET.
- Graph-based workflows: sequential, concurrent, handoff and group-chat patterns.
- Session state, checkpointing and human-in-the-loop built in.
- Multi-provider: Foundry, Azure OpenAI, OpenAI, Anthropic, Bedrock, Ollama.
- Open standards: MCP (tools), A2A (agent-to-agent), OpenAPI.
- Telemetry via OpenTelemetry; responsible-AI: task adherence, prompt shields, PII.

OPENAI SWARM → AGENTS SDK

The OpenAI path

What happened

- Swarm (Oct 2024) was a deliberately tiny, educational framework — agents + handoffs, nothing else.
- It had no state, tracing, guardrails or retries — never meant for production.
- March 2025: OpenAI shipped the Agents SDK as the production successor; Swarm's repo now redirects to it.
- Same mental model — the handoff pattern carries straight over.

Agents SDK — five primitives



Agents instructions, tools, model



Handoffs transfer control to a peer agent



Guardrails validate input & output at the edge



Sessions automatic conversation history



Tracing structured trace of every run

OpenAI-first, but runs on Azure OpenAI for EU data residency.

DECISION

Side-by-side: how they compare

	CrewAI	Agent Framework	Agents SDK	LangGraph
Model	Roles + tasks in a crew	Agents + graph workflows	Agents + handoffs	Typed state graph
Abstraction	High — role/YAML	Medium — SDK + workflow	Low — few primitives	Low — explicit graph
Multi-agent	Sequential / hierarchical	Group-chat · handoff · concurrent	Handoffs	Supervisor · swarm · subgraph
State / memory	Per-agent + Flow state	Session state + checkpoints	Sessions	Reducers + checkpointers
Azure path	Any model on Azure	Native: Foundry Agent Service	Via Azure OpenAI	Any via LangChain
Best fit	Business team workflows	Enterprise, regulated, MS stack	OpenAI-centric, fast handoffs	Complex, audited pipelines
Status	Active, standalone	GA — successor to AutoGen/SK	Active (Swarm retired)	Active, production-proven

No single winner — match the framework to the team, the control you need, and your Azure footprint.

DECISION

Which framework should you choose?



Choose CrewAI if...

- The task maps to a team of specialist roles
- Business users configure agents (YAML)
- You want fast iteration, little boilerplate
- Different agents need different LLMs



Choose Agent Framework if...

- You're on the Microsoft / Azure stack
- You need governance, telemetry, HITL
- Regulated industry (audit, finance, health)
- You want Foundry-native deployment



Choose Agents SDK if...

- You're OpenAI / Azure OpenAI centric
- Handoffs are your core pattern
- You want the fastest time-to-demo
- You want built-in tracing + guardrails

And you can combine them: an orchestrator in one framework can call a crew or an SDK agent as a tool.

03

AZURE EVENTING & SCALE-OUT

Service Bus between your agents



EVENTING

Why put a message broker between agents?



Decouple

Agents don't call each other directly — they exchange messages. Deploy and version each independently.



Temporal decoupling

Producer and consumer needn't be online together; messages wait durably in the queue.



Load-leveling

Traffic spikes fill the queue instead of overwhelming an agent; consumers drain at their own pace.



Scale-out

Add more consumer agents to the same queue — throughput rises with no code change.

Pick the right Azure messaging service

Service Bus — reliable business messaging: commands, transactions, agent workflows. **Event Grid** — lightweight event notifications. **Event Hubs** — high-volume telemetry / streaming.

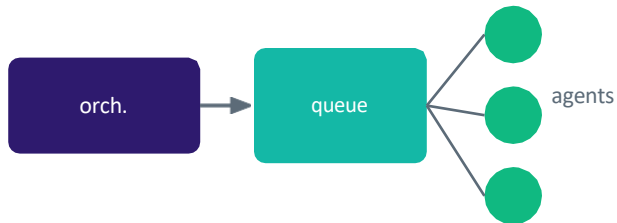
SERVICE BUS

Queues vs topics: dispatch vs broadcast



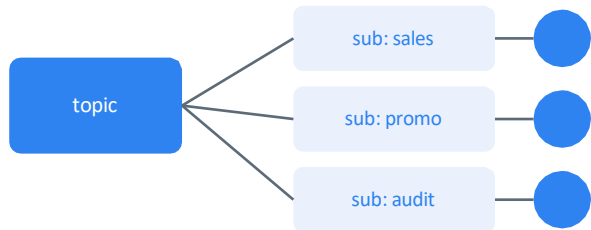
Queue — competing consumers

- Point-to-point: each message is processed by exactly one consumer.
- Many worker agents listen on one queue and compete — that IS horizontal scale-out.
- FIFO delivery; use for task / command dispatch to a pool of identical agents.



Topic + subscriptions — pub/sub

- One message is copied to every subscription (fan-out to many agents).
- Each subscription is a virtual queue with its own cursor and filter rules.
- Use to broadcast one event — a filter routes it to just the agents that care.



RELIABILITY

Sessions, peek-lock & dead-letter



Sessions

Group related messages by SessionId → delivered in order to one consumer holding an exclusive lock.

Ordered per-entity state; request-reply



Peek-lock

Two-stage receive: lock → process → complete. Crash before complete? The message reappears.

At-least-once; make handlers idempotent



Dead-letter (DLQ)

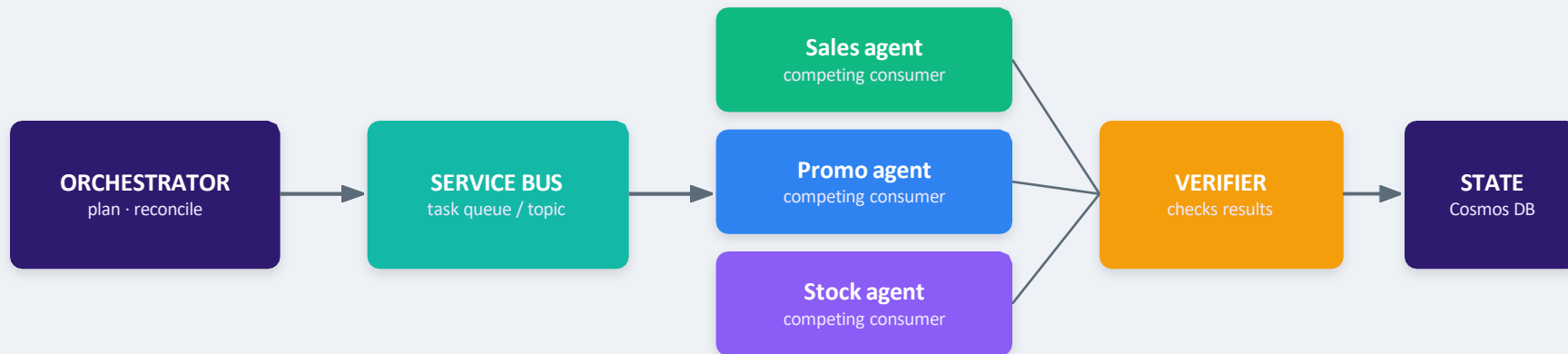
After MaxDeliveryCount failures a poison message is moved to a sidecar queue, not lost.

Quarantine + inspect + replay

These three turn 'agents chatting' into a reliable system: ordering when it matters, safe retries, and a place for poison to go.

BLUEPRINT

A multi-agent system on Azure



Models & tools: Azure AI Foundry / Azure OpenAI · identity: Managed Identity (Entra ID) · tracing: OpenTelemetry

Orchestrator dispatches tasks over Service Bus → specialist agents scale out as competing consumers → a verifier gates results → state persists in Cosmos DB.

04

FAILURE SCENARIOS

How multi-agent systems break



FAILURE MODES

Six ways a crew goes wrong



Cascading failure

One agent's error propagates through everything downstream that trusted it.



Duplicate work

No task registry → two agents grab the same task and produce conflicting outputs.



Agent drift

Over ~8–10 chained handoffs, intent, consensus and behaviour quietly degrade.



Poison / lost messages

A malformed message crashes a consumer on every retry, or a crash drops it entirely.



Orchestrator overload

A supervisor doing the hard reasoning becomes the bottleneck and single point of failure.



Groupthink / loops

Debate agents converge on a wrong answer, or never converge and loop forever.

MITIGATION

From failure mode to defence

Failure	Defence
Cascading failure	Circuit-breakers, bounded retries, timeouts; decouple via Service Bus so one agent can't take the rest down.
Agent drift	Prefer planner–worker over long handoff chains; re-ground each hop; add a verifier; cap handoff depth.
Orchestrator overload	Keep the orchestrator thin (route + reconcile); push reasoning to workers; checkpoint state for failover.
Duplicate work	Explicit task assignment; Service Bus peek-lock + sessions so one consumer owns a task; idempotency keys.
Poison / lost messages	Peek-lock (complete only on success); dead-letter after MaxDeliveryCount; idempotent, validated handlers.
Groupthink / loops	Differentiated roles; independent reasoning; a judge/verifier agent; hard cap on debate cycles.
Runaway cost	Model tiering (cheap sub-agents); token/budget ceilings via guardrails; profile per-run cost before scaling.

05

CHAOS DRILL

Break it on purpose, prove it recovers



CHAOS ENGINEERING

Inject failure to build confidence

The idea

- Deliberately inject failures in a controlled way to validate resilience — not wait for a real outage.
- It's hypothesis-driven: predict the steady state, break something, check the prediction.
- Always bound the blast radius — scope, safeguards, and permission to run.
- Origin: Netflix Chaos Monkey randomly killed production instances to force resilient design.



Azure Chaos Studio

- Managed fault injection across compute, network and app layers.
- Service-direct faults (control plane, no install) + agent-based faults (in-guest: CPU/memory pressure, kill process, network latency/packet loss).
- Experiment = steps (sequential) → branches (parallel) → actions (fault or wait).
- Built-in safeguards; wire into CI/CD and Azure Monitor to compare SLI vs SLO.

THE DRILL

A chaos drill for a multi-agent system



1 • Steady state

Define the SLO: e.g. all SKUs forecast, accuracy $\geq$ target, verifier pass-rate.



2 • Inject

Break one thing: kill an agent, slow a tool, poison or drop a message.



3 • Observe

Does it degrade gracefully — or cascade? Does the verifier / DLQ catch it?



4 • Measure & fix

Compare to the SLO, find the weak link, harden, and re-run.

Fault catalogue — and the safe behaviour you're verifying

Inject	Expected safe behaviour
Kill an agent (process kill)	Orchestrator degrades to a baseline / partial result and flags — it doesn't crash the run.
Slow a tool or LLM (latency)	Timeout fires; bounded retry; the graph doesn't hang the whole crew.
Poison a result (bad output)	Verifier rejects it; message dead-letters after MaxDeliveryCount — bad data never reaches the plan.
Drop / duplicate a message	Peek-lock redelivers; idempotency keys stop a duplicate work order.
Crash the orchestrator	Restart resumes from the last checkpoint instead of restarting the whole job.

06

INDUSTRY CASE STUDIES

Two real multi-agent deployments



CASE STUDY 1 · CPG

Demand planning at a global beverage maker

The problem

- Weekly demand forecasting stitched together SAP, Databricks and scattered Excel — slow and error-prone.
- Forecast accuracy hovered ~70%; 25–35% SKU error drives stockouts or bloated inventory.
- Five analysts spent ~40 hours a week just collating and cleaning data.
- Rule engines and RPA collapsed under multi-source complexity and exceptions.

70%

starting forecast accuracy

\$1.73T

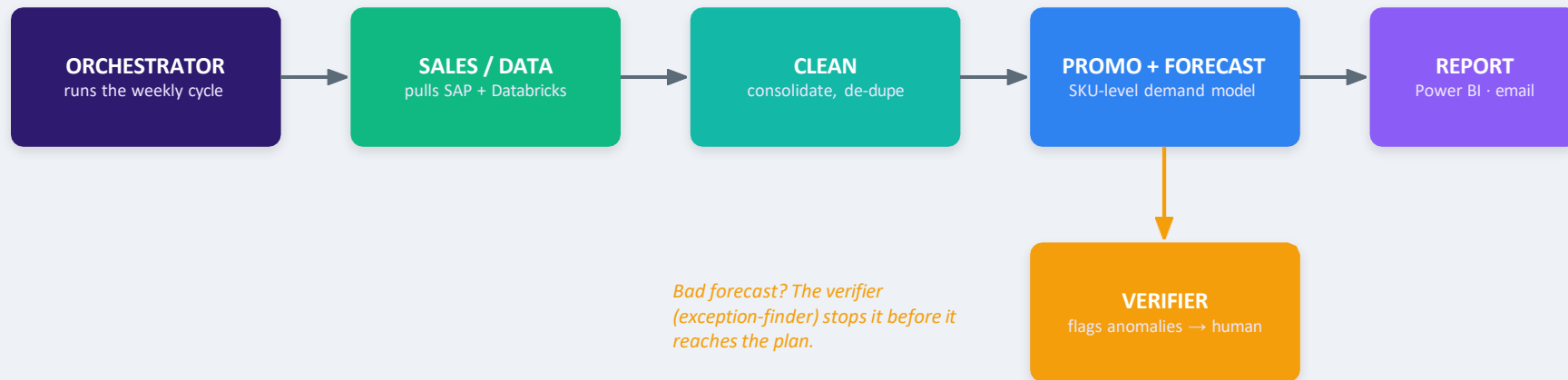
global retail inventory distortion

40 hrs/wk

manual work, 5 analysts

CASE STUDY 1 · CPG

The crew — and the verifier that guards it



Impact after the six-agent crew

~90%

automation of the cycle

week → minutes

to produce a forecast

1 month → 2 weeks

to deploy changes

30 min

to push model updates live

CASE STUDY 1 · CHAOS DRILL

Break the crew, watch it hold



Kill the promotions agent mid-cycle

Inject: Steady-state SLO: every SKU forecast. Inject: process-kill the promo agent.

Verify: Safe: orchestrator degrades to a baseline (no-promo) forecast and flags the gap — the run completes instead of crashing.



Poison one forecast

Inject: Inject a wildly off SKU forecast into the results queue.

Verify: Safe: the verifier rejects it; the message dead-letters after retries; the bad number never reaches the published plan.

A drill that passes is evidence, not a hope — you've shown the failure mode is contained before it hits production.

CASE STUDY 2 · REGULATED

KPMG — multi-agent audit on Azure

What they built

- KPMG Clara AI — the smart-audit platform used on KPMG audits worldwide, serving ~95,000 auditors.
- Underneath, KPMG Workbench coordinates a fleet of task-specific agents on Azure AI Foundry across audit, tax and advisory.
- Agents automate reconciliation, transaction testing and documentation; whole-dataset analysis replaces sampling.
- Built on Semantic Kernel, moving to Microsoft Agent Framework + Foundry Agent Service; grounded via Azure AI Search (RAG).

Why this pattern fits a regulated firm



Whole-dataset analysis, not sampling — higher coverage, earlier risk detection.



Human-in-the-loop and a Trusted-AI framework keep auditors in control.



Governance + observability in Azure AI Foundry — task adherence, PII, tracing.



Open standards: MCP for tools, A2A for agent-to-agent, across a global estate.

07

REFERENCE

Glossary of terms & jargon



GLOSSARY · 1 OF 3

Orchestration & patterns

Multi-agent orchestration Several role-specific agents coordinated on one complex task.

Orchestrator / supervisor The agent that plans, delegates, monitors and reconciles.

Sub-agent / worker A specialist that executes one bounded task and reports back.

Handoff One agent transfers the whole conversation to a peer agent.

Fan-out / fan-in Split work to parallel agents, then merge their results.

Consensus / debate Parallel agents reason independently; a rule or judge picks the answer.

Verifier / judge agent An agent whose only job is to check or rank others' output.

Subgraph A self-contained agent workflow embedded inside a larger one.

State-sharing How agents pass data: shared state, typed channels, structured output.

WBS Work-breakdown structure — the task decomposition the orchestrator plans.

Agent drift Quality decay across many chained handoffs (semantic/coordination).

HITL Human-in-the-loop — a person approves at a defined checkpoint.

GLOSSARY · 2 OF 3

Frameworks & SDKs

CrewAI Role-based framework: agents + tasks in a crew (sequential/hierarchical).

Crew / Flow CrewAI's team of agents+tasks; Flow is its orchestration layer.

AutoGen MS Research framework that pioneered group-chat multi-agent (now maintenance).

Microsoft Agent Framework Successor merging AutoGen + Semantic Kernel; Python/.NET, GA 2026.

Semantic Kernel Microsoft's enterprise agent SDK; folded into Agent Framework.

OpenAI Swarm Educational handoff framework (2024), now deprecated.

OpenAI Agents SDK Production successor to Swarm; Agents · Handoffs · Guardrails · Sessions · Tracing.

Guardrails Input/output validators that run before and after the model.

Sessions Automatic conversation-history management for an agent run.

Tracing A structured record of every agent step, tool call and handoff.

A2A Agent-to-Agent — an open protocol for agents to talk across frameworks.

MCP Model Context Protocol — a standard way for agents to discover and call tools.

Azure AI Foundry Azure's platform to build, deploy and govern agents (Agent Service).

LangGraph Graph-based framework for maximum control over stateful agent flows.

GLOSSARY · 3 OF 3

Azure eventing & chaos

Azure Service Bus Managed enterprise message broker: queues + pub/sub topics.

Queue Point-to-point; each message handled by exactly one consumer.

Topic / subscription Pub/sub: each subscription gets its own copy of a message.

Competing consumers Many workers on one queue — how you scale processing out.

Peek-lock Lock → process → complete; the message returns if you crash.

Session (SessionId) Ordered, exclusive delivery of related messages to one consumer.

Dead-letter queue (DLQ) Sidecar queue for poison messages after too many failures.

Idempotency Re-processing the same message causes no duplicate side-effect.

Correlation ID A shared ID that ties a chain of related messages for tracing.

Event Grid / Event Hubs Lightweight event notifications / high-volume streaming.

Chaos engineering Deliberately injecting failures to validate resilience.

Azure Chaos Studio Managed fault injection: agent-based & service-direct faults.

Steady-state hypothesis The measurable 'normal' a chaos experiment tests against.

Blast radius The bounded scope a chaos experiment is allowed to affect.

REFERENCE

Sources & further reading

-  **Microsoft Agent Framework — overview** learn.microsoft.com/agent-framework/overview
-  **Introducing Microsoft Agent Framework (AutoGen + SK)** azure.microsoft.com/blog
-  **OpenAI Agents SDK (successor to Swarm)** openai.github.io/openai-agents-python
-  **CrewAI — CPG demand forecasting case study** blog.crewai.com
-  **Azure Service Bus — queues, topics & subscriptions** learn.microsoft.com
-  **Competing Consumers pattern** learn.microsoft.com/azure/architecture
-  **Azure Chaos Studio — overview** learn.microsoft.com/azure/chaos-studio
-  **KPMG redefines audit with agentic AI on Azure** microsoft.com/customers

RECAP

Specialists, coordinated — and proven under failure

- Three topologies — orchestrator/sub-agent, parallel, handoffs — combine into most real systems.
- Match the framework to the team: CrewAI, Microsoft Agent Framework, or the OpenAI Agents SDK.
- Service Bus decouples and scales agents: queues for dispatch, topics for broadcast, sessions + DLQ for safety.
- Name the failure modes — cascade, drift, duplication, poison, groupthink — and mitigate each by design.
- Run the chaos drill: break one agent on purpose and prove the crew degrades gracefully.